

RUNBOOK TÉCNICO · ROBÓTICA & TELEOPERACIÓN

Robot Humanoide G1 con unitree_rl_gym

Búsqueda y aproximación autónoma a objetos por color, simulada en MuJoCo (Sim2Sim) como paso previo al despliegue en el robot físico (Sim2Real).

1 Por qué cambiamos de framework

Hasta este punto el proyecto usaba **unitree_rl_lab**, un framework de Unitree con una política de control entrenada en Isaac Lab. Funcionaba, pero el robot se balanceaba/temblaba visiblemente al caminar, incluso después de ajustar la amortiguación (damping) de las articulaciones.

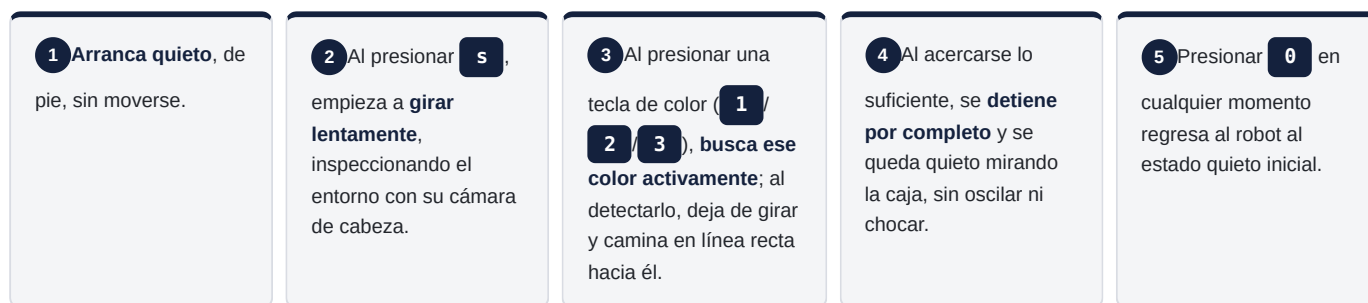
Unitree publica un segundo framework, **unitree_rl_gym**, con una política de caminata preentrenada distinta (entrenada con Isaac Gym / legged_gym) que resultó ser notablemente más estable de fábrica — *"casi sin temblor visible"* en las primeras pruebas, sin necesidad de tocar ganancias ni amortiguación. Por eso migramos:

- **unitree_rl_lab**: modelo G1 de 29 grados de libertad (incluye brazos y torso articulados), política más "nerviosa".
- **unitree_rl_gym**: modelo G1 de 12 grados de libertad (solo piernas; brazos y torso son geometría fija), política de caminata mucho más estable, pensada como referencia oficial de Unitree para caminar y luego pasar a Sim2Real.

Plan explícito del proyecto: primero validar todo en simulación pura MuJoCo (Sim2Sim, script Python [deploy_mujoco.py](#)), y sólo después de tenerlo estable, pasar a despliegue en el robot físico (Sim2Real). Ese paso a Sim2Real todavía no se ha realizado.

2 Qué hace la aplicación (objetivo general)

Se colocan tres cajas de colores (roja, verde, azul) alrededor del robot en la simulación. El robot debe seguir esta secuencia:



Todo esto corre sobre la política de caminata preentrenada de **unitree_rl_gym**: el robot nunca deja de "caminar" con su controlador RL normal; lo único que cambia es el comando de velocidad lineal/angular que se le inyecta según lo que ve la cámara.

3 Arquitectura del sistema

- **Simulador:** MuJoCo, cargando `resources/robots/g1_description/scene.xml` (modelo de 12 DoF).
- **Cámara:** un `<camera name="head_camera">` agregado al `<body name="pelvis">` del modelo, orientada hacia abajo (`xyaxes="0 -1 0 0.500 0 0.866"`, `fovy="58"`) para ver el suelo frente al robot.
- **Objetos de color:** tres cajas (`target_red`, `target_green`, `target_blue`) agregadas a `scene.xml`, ubicadas alrededor del robot en la dirección que cubre la cámara.
- **Script principal:** `deploy/deploy_mujoco/deploy_mujoco_vision.py` (copia modificada del `deploy_mujoco.py` oficial).

Hilo de visión

Un hilo (`vision_and_viewer_thread`) renderiza la cámara del robot en cada frame, hace detección de color por umbral RGB (rojo/verde/azul), calcula el error horizontal (qué tan centrado está el color en la imagen) y una posición vertical usada como referencia de distancia, y abre una ventana ("lo que ve el robot") donde además se capturan las teclas **S**, **0**, **1**, **2**, **3**.

Lógica de navegación — `compute_cmd_from_vision()`

Traduce el estado de visión en el comando `[vx, vy, wz]` enviado a la política de RL en cada paso de control:

- `target == -1` → comando cero (quieto).

- `target == 0` , o color elegido pero aún no detectado → gira en el sitio a velocidad de escaneo.
- Color detectado → suaviza el error horizontal con un filtro exponencial (evita giros bruscos por ruido de una sola imagen) y avanza en línea recta mientras el error esté centrado.
- Al llegar a la distancia mínima, se activa un "**candado**" de llegada que congela el comando en cero de forma permanente hasta que cambie el objetivo.

4 Ajustes realizados en esta sesión

Ajuste	Antes	Ahora	Motivo
Límite de tiempo de simulación (<code>simulation_duration</code> en <code>configs/gl.yaml</code>)	60.0 s	999999.0 s	El script se cerraba solo a los 60 s, cortando la secuencia de interacción a la mitad. Con este valor la ventana permanece abierta hasta cierre manual.
Velocidad de avance (<code>lin_vel_x</code> , línea ~163)	0.5 m/s	0.65 m/s	Robot más ágil al aproximarse, sin perder estabilidad de la política preentrenada.
Distancia de las cajas de color (<code>scene.xml</code>)	~1.7–1.8 m	~2.8 m	Recorrido de búsqueda y aproximación más largo y visible, manteniendo el mismo ángulo respecto al robot.

Coordenadas finales de las cajas

```
<body name="target_red"    pos="2.800  0.000 0.15"> ... </body>
<body name="target_green"  pos="0.161  2.796 0.15"> ... </body>
<body name="target_blue"   pos="0.161 -2.796 0.15"> ... </body>
```

5 Cómo iniciar la aplicación

```
# 1. Asegúrate de tener VcXsrv (XLaunch) abierto en Windows y el DISPLAY configurado
export DISPLAY=localhost:0

# 2. Ve a la carpeta del proyecto
cd ~/unitree_rl_gym

# 3. Ejecuta el script de despliegue en MuJoCo (Sim2Sim) con visión y navegación por color
PYTHONPATH=~/.unitree_rl_gym python3 deploy/deploy_mujoco/deploy_mujoco_vision.py gl.yaml
```

Esto abre dos ventanas: el visor de MuJoCo (el robot y las cajas de colores) y la ventana "lo que ve el robot" (vista de la cámara de cabeza, usada también para capturar las teclas).

Secuencia de control

Tecla	Acción
s	Empieza a girar en el sitio, inspeccionando el entorno
1	Busca y camina hacia la caja roja
2	Busca y camina hacia la caja verde
3	Busca y camina hacia la caja azul
0	Regresa al estado quieto (cancela búsqueda/aproximación en curso)
q	Cierra el visor (en la ventana de la cámara)

Comportamiento esperado: el robot arranca de pie y completamente quieto; solo se mueve al recibir la orden , y solo camina en línea recta una vez que detecta el color solicitado.